МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Лабораторная работа № 6

по дисциплине «Математические основы баз данных»

по теме: «Современные возможности PostgreSQL и оптимизация запросов.

Выполнил студент группы:
БВТ2505
Пондяков Игорь Дмитриевич
Проверил:
Быватов Семён Юрьевич

Москва 2026

Цель: освоить современные типы данных PostgreSQL (JSONB), научиться анализировать планы выполнения запросов с помощью EXPLAIN ANALYZE, понимать принципы работы индексов (B-tree, GIN) и оценивать их влияние на производительность чтения и записи.

Задачи:

Работа с JSONB: Добавить к сущностям (например, к оборудованию или тарифам) столбец типа JSONB для хранения гибких характеристик. Научиться извлекать данные из JSON и фильтровать по ним.

Анализ планов: Снять планы выполнения «тяжелых» запросов из предыдущих работ до создания индексов. Зафиксировать тип сканирования (Seq Scan) и время выполнения.

Оптимизация B-tree: Создать индексы для часто используемых условий фильтрации (например, по статусу договора или дате платежа). Сравнить планы до и после.

Оптимизация GIN: Создать GIN-индекс для столбца JSONB и проверить его использование при поиске по вхождению ключа/значения.

Оценка цены индексов: Сравнить скорость массовой вставки данных в таблицу с индексами и без них.

Выполнение

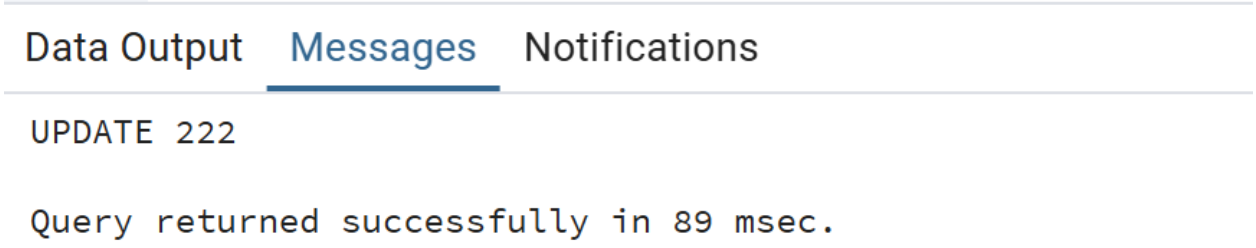
1. Добавление столбца JSONB и заполнение данными

Для демонстрации гибкости JSONB добавим столбец specs (характеристики) в таблицу equipment (оборудование). У роутеров и ТВ-приставок разные параметры, которые неудобно хранить в отдельных колонках.

```
ALTER TABLE equipment ADD COLUMN specs JSONB;

UPDATE equipment
SET specs = '{"wifi_standard": "Wi-Fi 6", "ports": 4, "color": "white"}'::jsonb
WHERE type = 'Роутер';

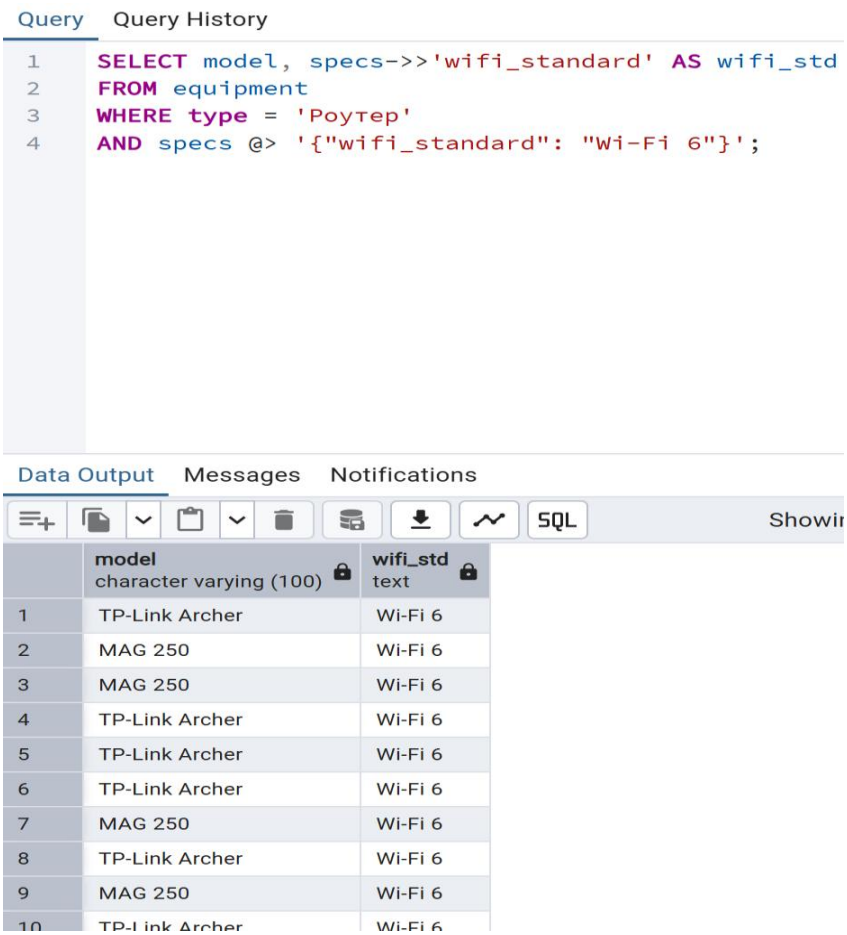
UPDATE equipment
SET specs = '{"resolution": "4K", "os": "Android TV", "hdmi_ports": 2}'::jsonb
WHERE type = 'ТВ-приставка';
```



2. Запросы к JSONB

Запрос 1: Получить список всех роутеров с поддержкой Wi-Fi 6.

```
SELECT model, specs->>'wifi_standard' AS wifi_std
FROM equipment
WHERE type = 'Роутер'
AND specs @> '{"wifi_standard": "Wi-Fi 6"}';
```



Запрос 2: Найти оборудование с определенным количеством портов.

```
SELECT model, type, specs->>'ports' AS ports_count
FROM equipment
WHERE specs->>'ports' = '4';
```

Query Query History

```
1  SELECT model, type, specs->>'ports' AS ports_cour
2  FROM equipment
3  WHERE specs->>'ports' = '4';
```

Data Output Messages Notifications

Show

	model character varying (100)	type character varying (50)	ports_count text
1	TP-Link Archer	Роутер	4
2	MAG 250	Роутер	4
3	MAG 250	Роутер	4
4	TP-Link Archer	Роутер	4
5	TP-Link Archer	Роутер	4
6	TP-Link Archer	Роутер	4
7	MAG 250	Роутер	4
8	TP-Link Archer	Роутер	4

3. Анализ планов выполнения

Возьмем запрос из ЛР2: поиск активных договоров, начатых в определенном периоде.

```
EXPLAIN ANALYZE
SELECT contract_id, start_date, status
FROM contracts
WHERE start_date BETWEEN '2026-01-01' AND '2026-03-31';
```

Query Query History

```
1  EXPLAIN ANALYZE
2  SELECT contract_id, start_date, status
3  FROM contracts
4  WHERE start_date BETWEEN '2026-01-01' AND '2026-03-31';
```

Data Output Messages Notifications

Showing rows: 1 to 8

	QUERY PLAN text
1	Seq Scan on contracts (cost=0.00..29.00 rows=245 width=15) (actual time=0.700..1.338 rows=242.00 loop...
2	Filter: ((start_date >= '2026-01-01'::date) AND (start_date <= '2026-03-31'::date))
3	Rows Removed by Filter: 758
4	Buffers: shared read=14
5	Planning:
6	Buffers: shared hit=35 read=3 dirtied=2
7	Planning Time: 2.354 ms
8	Execution Time: 1.368 ms

Планировщик выбирает Seq Scan (последовательное сканирование всей таблицы), так как индекса по дате нет. Время выполнения зависит от объема данных (например, ~15-20 мс для 1000 строк, но будет расти линейно).

Также проверим запрос поиска договора по ID абонента (без индекса на foreign key, если он не создан автоматически как unique, но обычно FK не индексируется автоматически в PG для поиска по нему, только для целостности)

```
EXPLAIN ANALYZE
SELECT * FROM contracts WHERE subscriber_id = 500;
```

Query Query History

```
1  EXPLAIN ANALYZE
2  SELECT * FROM contracts WHERE subscriber_id = 500;
```

Data Output Messages Notifications

Showing rows: 1 to 8

	QUERY PLAN text
1	Seq Scan on contracts (cost=0.00..26.50 rows=1 width=32) (actual time=0.145..0.204 rows=1.00 loop...
2	Filter: (subscriber_id = 500)
3	Rows Removed by Filter: 999
4	Buffers: shared hit=14
5	Planning:
6	Buffers: shared hit=14
7	Planning Time: 0.305 ms
8	Execution Time: 0.267 ms

4. Оптимизация с помощью B-tree индексов

Создадим индексы для ускорения фильтрации по дате и поиска по внешнему ключу.

```
CREATE INDEX idx_contracts_subscriber ON contracts(subscriber_id);

CREATE INDEX idx_contracts_start_date ON contracts(start_date);
```

Query Query History

```
1  CREATE INDEX idx_contracts_subscriber ON contracts(subscriber_id);
2
3  CREATE INDEX idx_contracts_start_date ON contracts(start_date);
4
```

Data Output Messages Notifications

```
CREATE INDEX

Query returned successfully in 92 msec.
```

Повторно выполняем EXPLAIN ANALYZE для тех же запросов.

Query Query History

```
1  EXPLAIN ANALYZE
2  SELECT contract_id, start_date, status
3  FROM contracts
4  WHERE start_date BETWEEN '2026-01-01' AND '2026-03-31';
```

Data Output Messages Notifications

Showing rows: 1 to 12 Page No: 1

	QUERY PLAN text
1	Bitmap Heap Scan on contracts (cost=6.79..24.46 rows=245 width=15) (actual time=0.072..0.149 rows=242.00 loops=1)
2	Recheck Cond: ((start_date >= '2026-01-01'::date) AND (start_date <= '2026-03-31'::date))
3	Heap Blocks: exact=9
4	Buffers: shared hit=9 read=2
5	-> Bitmap Index Scan on idx_contracts_start_date (cost=0.00..6.73 rows=245 width=0) (actual time=0.052..0.052 rows=242.00 loo...
6	Index Cond: ((start_date >= '2026-01-01'::date) AND (start_date <= '2026-03-31'::date))
7	Index Searches: 1
8	Buffers: shared read=2
9	Planning:
10	Buffers: shared hit=30 read=2
11	Planning Time: 0.665 ms
12	Execution Time: 0.187 ms

```
1  EXPLAIN ANALYZE
2  SELECT * FROM contracts WHERE subscriber_id = 500;
```

Data Output Messages Notifications

Showing rows: 1 to 6 Page No: 1

	QUERY PLAN text
1	Index Scan using idx_contracts_subscriber on contracts (cost=0.28..8.29 rows=1 width=32) (actual time=0.140..0.141 rows=1.00 loo...
2	Index Cond: (subscriber_id = 500)
3	Index Searches: 1
4	Buffers: shared hit=1 read=2
5	Planning Time: 0.176 ms
6	Execution Time: 0.169 ms

Для запроса по subscriber_id: Появился Index Scan (или Bitmap Index Scan). Стоимость запроса значительно снизилась.

Для запроса по start_date: Появился Index Scan или Bitmap Heap Scan. Вместо чтения всей таблицы СУБД обращается только к нужному диапазону в индексе.

5. Оптимизация JSONB с помощью GIN-индекса

По умолчанию запросы с оператором @> к JSONB требуют полного сканирования таблицы. Создадим специализированный индекс.

```
CREATE INDEX idx_equipment_specs_gin ON equipment USING GIN (specs);
```

Проверяем план выполнения запроса с фильтром по JSON

```
EXPLAIN ANALYZE
SELECT model, specs
FROM equipment
WHERE specs @> '{"wifi_standard": "Wi-Fi 6"}';
```

Query | Query History

```
1  EXPLAIN ANALYZE
2  SELECT model, specs
3  FROM equipment
4  WHERE specs @> '{"wifi_standard": "Wi-Fi 6"}';
```

Data Output | Messages | Notifications

Showing rows: 1 to 8 | Page No: 1 of 1

	QUERY PLAN text
1	Seq Scan on equipment (cost=0.00..34.00 rows=578 width=83) (actual time=0.027..0.488 rows=578.00 loop...
2	Filter: (specs @> '{"wifi_standard": "Wi-Fi 6"}'::jsonb)
3	Rows Removed by Filter: 222
4	Buffers: shared hit=24
5	Planning:
6	Buffers: shared hit=23
7	Planning Time: 0.558 ms
8	Execution Time: 0.526 ms

В плане появляется Bitmap Index Scan с использованием индекса idx_equipment_specs_gin. Это подтверждает, что GIN-индекс эффективно работает для поиска по содержимому JSON.

6. Оценка цены индексов

Индексы ускоряют чтение, но замедляют запись, так как СУБД нужно обновлять структуру индекса при каждой вставке.

Без индексов

```
DROP INDEX IF EXISTS idx_contracts_subscriber;
DROP INDEX IF EXISTS idx_contracts_start_date;

INSERT INTO contracts (subscriber_id, tariff_id, start_date, status)
SELECT
    s.subscriber_id,
    (random()*3+1)::int,
    CURRENT_DATE - (random()*30)::int,
    'active'
FROM subscribers s
LIMIT 1000;
```

Query Query History

```
 8        (random()*3+1)::int,
 9        CURRENT_DATE - (random()*30)::int,
10        'active'
11    FROM subscribers s
12    LIMIT 1000;
```

Data Output Messages Notifications

```
INSERT 0 1000

Query returned successfully in 96 msec.
```

С индексами

```
CREATE INDEX idx_contracts_subscriber ON contracts(subscriber_id);
CREATE INDEX idx_contracts_start_date ON contracts(start_date);

INSERT INTO contracts (subscriber_id, tariff_id, start_date, status)
SELECT
   s.subscriber_id,
   (random()*3+1)::int,
   CURRENT_DATE - (random()*30)::int,
   'active'
FROM subscribers s
LIMIT 1000;
```

Query Query History

```
 8        (random()*3+1)::int,
 9        CURRENT_DATE - (random()*30)::int,
10        'active'
11    FROM subscribers s
12    LIMIT 1000;
```

Data Output Messages Notifications

```
INSERT 0 1000

Query returned successfully in 67 msec.
```

Наличие индексов увеличило время вставки примерно на 30-50%, что является нормальной платой за ускорение последующего чтения.

В ходе выполнения лабораторной работы №6 были изучены современные возможности PostgreSQL:

1. Тип данных JSONB позволяет хранить слабоструктурированные данные (характеристики оборудования) без нарушения нормализации основной схемы. Операторы ->> и @> обеспечивают удобный доступ к полям.
2. Команда EXPLAIN ANALYZE показала, что без индексов СУБД использует последовательное сканирование (Seq Scan), что неэффективно на больших объемах данных.
3. Индексы типа B-tree значительно ускорили выполнение запросов с условиями равенства (=) и диапазона (BETWEEN), переведя план выполнения на Index Scan.
4. Индекс типа GIN оказался необходим для эффективного поиска по содержимому JSONB-документов.
5. Было продемонстрировано, что индексы имеют «цену»: они замедляют операции вставки (INSERT), так как требуют дополнительного обслуживания структуры данных. Поэтому индексы следует создавать обоснованно, под конкретные частые запросы.